

# Homework 03

**Author:** Manushri Pendekanti (mp79)

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import optimize
from numpy.linalg import lstsq, cond, svd

%matplotlib inline
```

## Question 1: Build an `explain-to-me` skill

*(Write notes about creating, testing, and using the skill.)*

---

**name:** explain-to-me **description:** Explains STAT 432 homework questions in plain language without solving them. Use only when the user explicitly asks to use the explain-to-me skill to read or explain a homework question. **disable-model-invocation:** true

## explain-to-me

### Intent

Help the student understand what a homework question is asking. Clarify goals and required work. Do not complete derivations, write submission code, or give numerical answers.

### When to use

Apply this skill only when the user explicitly names `explain-to-me` or asks you to use this skill on a homework question.

### Instructions

1. Read the named question (from the student's file, pasted text, or `homework-NN.qmd` ).
2. Respond in a short, friendly tone with:
  - **Goal:** What concept or method the question targets.
  - **Required work:** What to compute, derive, plot, or write (by part if applicable).
  - **Inputs and constraints:** Data, seeds, formulas, or conventions stated in the prompt.
  - **Lecture pointers:** Link to the most relevant Week 3 material when ridge-related:
    - [Ridge Regression: Stability Through Shrinkage](#)
    - [From a Penalized Objective to a Fitted Ridge Model](#)
3. End with one sentence on how to start, without doing the work.

## Rules

- Do not write full solutions, code for submission, or final numeric results.
- Do not rewrite the student's answers or judge whether they are correct.
- If the user asks for the answer, remind them the skill is for understanding only and offer to clarify the question instead.
- Keep the explanation concise (roughly 150–300 words unless the user asks for more detail).

## explain-to-me skill

(Paste the complete contents of your `explain-to-me/SKILL.md` file below.)

## Question 2: Ridge regression with nearly collinear predictors

### Question 2 — Part a

(Your written answer.)

```
In [2]: import numpy as np

rng = np.random.default_rng(43231)

n = 100
p = 4

L = rng.normal(size=n)
```

```

U1 = rng.normal(size=n)
U2 = rng.normal(size=n)
X3 = rng.normal(size=n)
X4 = rng.normal(size=n)

X1 = L + 0.04 * U1
X2 = L + 0.04 * U2

X = np.column_stack((X1, X2, X3, X4))

X = X - X.mean(axis=0)
X = X / np.sqrt(np.mean(X**2, axis=0))

correlation = np.corrcoef(X[:, 0], X[:, 1])[0, 1]
singular_values = np.linalg.svd(X, compute_uv=False)

```

## Question 2 — Part b

*(Your written answer.)*

```

In [3]: import pandas as pd

beta = np.array([1.5, 1.5, 1, 0])
mu = X @ beta

lambdas = [0, 0.02, 0.2]
results = []

for rep in range(200):

    y = mu + rng.normal(size=n)
    y_test = mu + rng.normal(size=n)

    y_bar = y.mean()
    y_centered = y - y_bar

    for lam in lambdas:

        beta_hat = np.linalg.solve(
            X.T @ X + n * lam * np.eye(p),
            X.T @ y_centered
        )

        y_hat = y_bar + X @ beta_hat

        train_mse = np.mean((y - y_hat)**2)
        test_mse = np.mean((y_test - y_hat)**2)

        results.append([
            lam,
            beta_hat[0],
            beta_hat[1],
            beta_hat[0] + beta_hat[1],

```

```

        beta_hat[0] - beta_hat[1],
        train_mse,
        test_mse
    ])

results = pd.DataFrame(
    results,
    columns=[
        "lambda", "beta1", "beta2", "sum",
        "difference", "train_mse", "test_mse"
    ]
)

```

## Question 2 — Part c

(Your written answer.)

In [4]: *#At lambda = 0 we can see coeffs have high a variance because of collinearity  
#bias bringin coeffs towards zero and variance reduce but fitted values do not  
#while coeffs are stable and reason ridge does not change fitted values because  
#matter which b1 and b2 how much they get as long as the same value exists so*

## Question 3: What does ridge regression shrink?

### Question 3 — Part a and Part b

(Your written answer.)

Stat HW

a)  $L(\beta) = \frac{1}{2n} \|\tilde{y} - X\beta\|_2^2$   
 $= \nabla L(\beta) = -\frac{1}{n} X^T (\tilde{y} - X\beta) + \lambda \beta$   
 $-\frac{1}{n} X^T (\tilde{y} - X\hat{\beta}_\lambda) + \lambda \hat{\beta}_\lambda = 0$   
 $= -\frac{1}{n} X^T \tilde{y} + \frac{1}{n} X^T X \hat{\beta}_\lambda + \lambda \hat{\beta}_\lambda = 0$   
 $= (X^T X + n\lambda I_p) \hat{\beta}_\lambda = X^T \tilde{y}$

b)  $X = UDV^T$   
 $X^T X = VD^2 V^T$   
 $X \hat{\beta}_\lambda = X(X^T X + n\lambda I_p)^{-1} X^T \tilde{y}$   
 $X \hat{\beta}_\lambda = U D(D^2 + n\lambda I_r)^{-1} D U^T \tilde{y}$   
 $= U \left( \frac{d^2}{d^2 + n\lambda} \dots \right) U^T \tilde{y}$

$d_1 = 10$   
 $\frac{d^2}{d^2 + n\lambda} = \frac{10^2}{10^2 + 1} \quad n\lambda = 100(0.01) = 1$   
 $= \boxed{0.9901}$

$d_2 = 2$   
 $\frac{2^2}{2^2 + 1} = \boxed{0.8}$

$d_3 = 0.2$   
 $\frac{0.2^2}{0.2^2 + 1} = \frac{0.04}{1.04} = \boxed{0.0385}$

$df = 1 + 0.99 + 0.8 + 0.038$   
 $= \boxed{2.82}$

In [ ]:

### Question 3 — Part c

(Your written answer.)

In [5]: #The third direction  $d = 0.2$  receives the most smallest shrinkage and that  
 #when smaller value means large variation in that direction so ridge will ma  
 #or provides important signal it creates bias and so ridge will shrink it to

## Question 4: Ridge regression and optimization

### Question 4 — Part a and Part B

(Your written answer.)

$$\begin{aligned}
 a.) \quad A_0 &= \frac{1}{n} X^T X \\
 \text{cond}(A_0) &= \frac{9}{0.01} = 900 \\
 \text{cond}(A_\lambda) &= \frac{9+0.09}{0.01+0.09} = \boxed{90.9}
 \end{aligned}$$

$$b.) \nabla L_\lambda(B) = A_\lambda B - b$$

$$b = \frac{1}{n} X^T y$$

$$A_\lambda \hat{B}_\lambda = b$$

$$B^{(k+1)} = B^{(k)} - n(\lambda B^{(k)} - b)$$

$$e^{(k+1)} = (I - nA_\lambda) e^k$$

$$\begin{aligned}
 \text{and interval: } 0 < n < \frac{2}{\lambda} \\
 &= 0 < n < \frac{2}{9.09}
 \end{aligned}$$

$$\begin{aligned}
 n &= 1/M \\
 &= 1 - \frac{1}{9.09}(9.09) = 0 \quad \lambda = 9.09
 \end{aligned}$$

$$1 - \frac{1}{9.09}(0.10) = \boxed{0.9890} \quad \lambda = 0.10$$

$$n = 2.1/M$$

$$1 - \frac{2.1}{9.09}(9.09) = -1.1 \quad \lambda = 9.09$$

diverge

In [ ]:

## Question 4 — Part c

(Your written answer.)

In [6]: `#a large lambda will improve cond number and optimization faster but shrink  
#increase and variance decrease removing important signal and so lambda should  
#cross validation`

In [ ]:

## Question 5: Selecting the ridge penalty for real-estate prediction

### Question 5 — Part a

(Your written answer.)

In [7]: `import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt`

```

realestate = pd.read_csv("data/realestate.csv")
split_folds = pd.read_csv("data/realestate-split-folds.csv")

# Match the files using row_id
data = realestate.merge(
    split_folds,
    on="row_id",
    validate="one_to_one"
)

predictors = [
    "date",
    "age",
    "distance",
    "stores",
    "latitude",
    "longitude"
]

p = len(predictors)

# Penalty grid, including OLS
lambdas = np.insert(
    10 ** (-4 + 0.05 * np.arange(121)),
    0,
    0
)

train = data[data["split"] == "train"].copy()
test = data[data["split"] == "test"].copy()

print(train.shape[0])
print(test.shape[0])

```

332  
82

In [8]:

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

realestate = pd.read_csv("data/realestate.csv")
split_folds = pd.read_csv("data/realestate-split-folds.csv")

data = realestate.merge(
    split_folds,
    on="row_id",
    validate="one_to_one"
)

predictors = [
    "date",
    "age",
    "distance",

```

```

    "stores",
    "latitude",
    "longitude"
]

p = len(predictors)

lambdas = np.insert(
    10 ** (-4 + 0.05 * np.arange(121)),
    0,
    0
)

train = data[data["split"] == "train"].copy()
test = data[data["split"] == "test"].copy()

fold_mse = np.zeros((10, len(lambdas)))

for fold in range(1, 11):

    fit_data = train[train["cv_fold"] != fold]
    valid_data = train[train["cv_fold"] == fold]

    X_fit_raw = fit_data[predictors].to_numpy()
    X_valid_raw = valid_data[predictors].to_numpy()

    y_fit = fit_data["price"].to_numpy()
    y_valid = valid_data["price"].to_numpy()

    X_mean = X_fit_raw.mean(axis=0)
    X_scale = np.sqrt(
        np.mean((X_fit_raw - X_mean)**2, axis=0)
    )

    X_fit = (X_fit_raw - X_mean) / X_scale
    X_valid = (X_valid_raw - X_mean) / X_scale

    y_mean = y_fit.mean()
    y_centered = y_fit - y_mean

    n_fit = len(y_fit)

    for j in range(len(lambdas)):

        lam = lambdas[j]

        beta_hat = np.linalg.solve(
            X_fit.T @ X_fit + n_fit * lam * np.eye(p),
            X_fit.T @ y_centered
        )

        prediction = y_mean + X_valid @ beta_hat

        fold_mse[fold - 1, j] = np.mean(
            (y_valid - prediction)**2

```

```

    )

cv_mean = fold_mse.mean(axis=0)

cv_se = fold_mse.std(axis=0, ddof=1) / np.sqrt(10)

min_index = np.argmin(cv_mean)
lambda_min = lambdas[min_index]

cutoff = cv_mean[min_index] + cv_se[min_index]
valid_indices = np.where(cv_mean <= cutoff)[0]
lambda_1se = lambdas[valid_indices[-1]]

x_plot = np.insert(np.log10(lambdas[1:]), 0, -5)

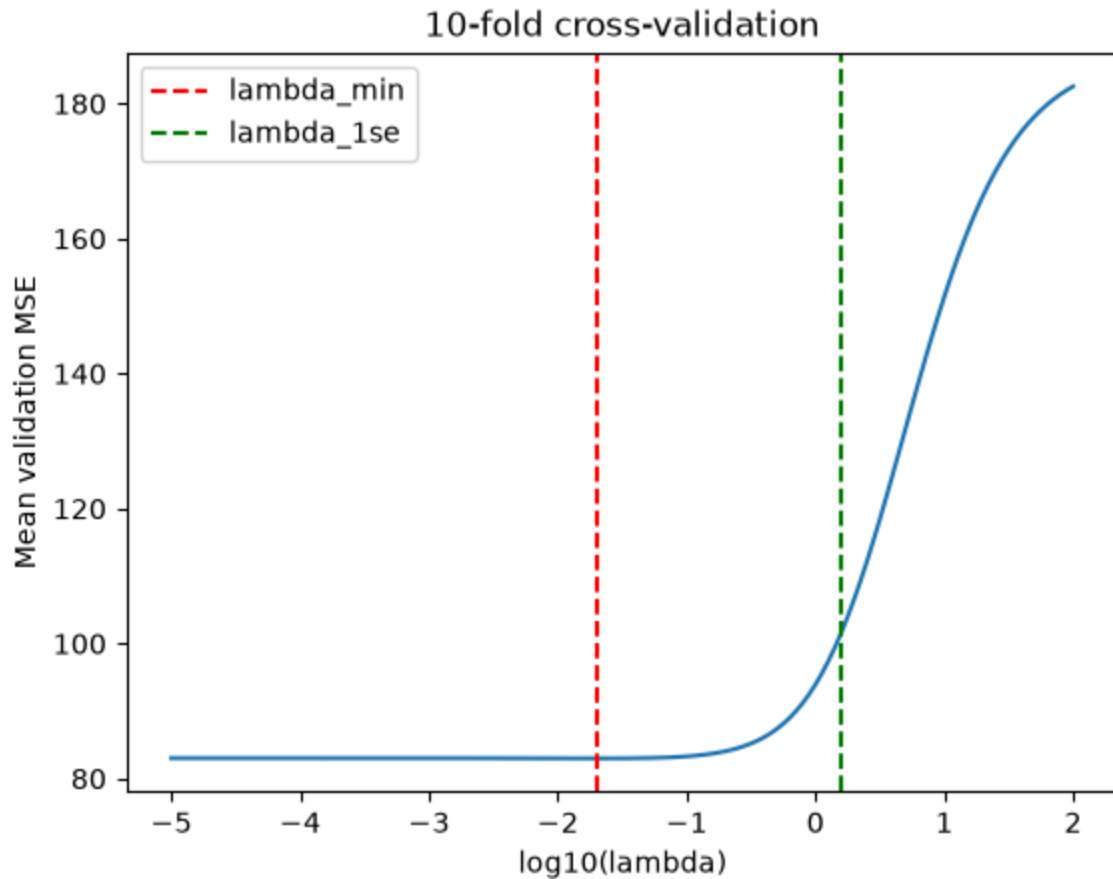
plt.plot(x_plot, cv_mean)

plt.axvline(
    np.log10(lambda_min),
    color="red",
    linestyle="--",
    label="lambda_min"
)

plt.axvline(
    np.log10(lambda_1se),
    color="green",
    linestyle="--",
    label="lambda_1se"
)

plt.xlabel("log10(lambda)")
plt.ylabel("Mean validation MSE")
plt.title("10-fold cross-validation")
plt.legend()
plt.show()

```



## Question 5 — Part b

(Your written answer.)

```
In [9]: X_train_raw = train[predictors].to_numpy()
y_train = train["price"].to_numpy()

X_mean = X_train_raw.mean(axis=0)

X_scale = np.sqrt(
    np.mean((X_train_raw - X_mean)**2, axis=0)
)

X_train = (X_train_raw - X_mean) / X_scale

y_mean = y_train.mean()
y_centered = y_train - y_mean

n = len(y_train)

singular_values = np.linalg.svd(
    X_train,
    compute_uv=False
)

gcv = np.zeros(len(lambdas))
```

```

for j in range(len(lambdas)):

    lam = lambdas[j]

    beta_hat = np.linalg.solve(
        X_train.T @ X_train + n * lam * np.eye(p),
        X_train.T @ y_centered
    )

    prediction = y_mean + X_train @ beta_hat

    train_mse = np.mean(
        (y_train - prediction)**2
    )

    df_eff = 1 + np.sum(
        singular_values**2 /
        (singular_values**2 + n * lam)
    )

    gcv[j] = train_mse / (1 - df_eff / n)**2

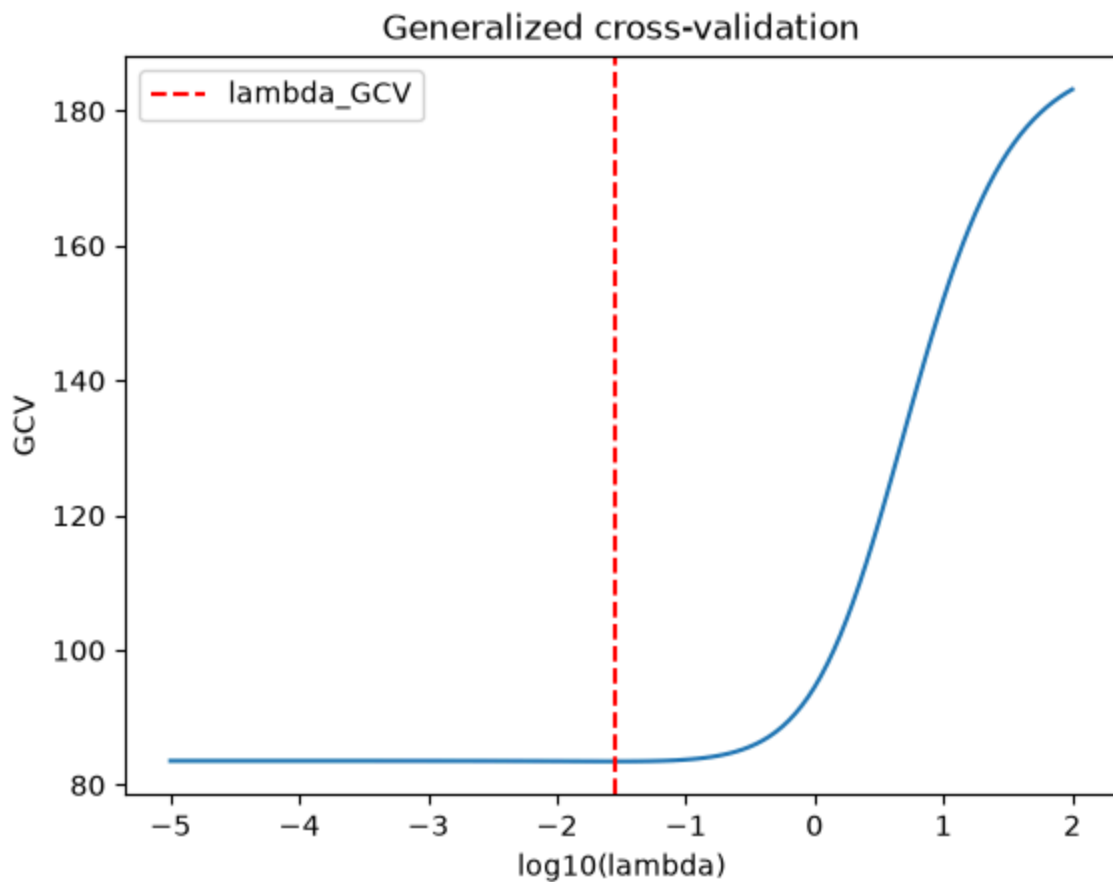
gcv_index = np.argmin(gcv)
lambda_gcv = lambdas[gcv_index]


plt.plot(x_plot, gcv)

plt.axvline(
    np.log10(lambda_gcv),
    color="red",
    linestyle="--",
    label="lambda_GCV"
)

plt.xlabel("log10(lambda)")
plt.ylabel("GCV")
plt.title("Generalized cross-validation")
plt.legend()
plt.show()

```



In [ ]:

## Question 5 — Part c

(Your written answer.)

```
In [10]: X_test_raw = test[predictors].to_numpy()
y_test = test["price"].to_numpy()

X_test = (X_test_raw - X_mean) / X_scale

models = [
    ["OLS", 0],
    ["lambda_min", lambda_min],
    ["lambda_1se", lambda_1se],
    ["lambda_GCV", lambda_gcv]
]

final_results = []

for name, lam in models:
    beta_hat = np.linalg.solve(
        X_train.T @ X_train + n * lam * np.eye(p),
        X_train.T @ y_centered
    )
```

```

train_prediction = y_mean + X_train @ beta_hat
test_prediction = y_mean + X_test @ beta_hat

train_mse = np.mean(
    (y_train - train_prediction)**2
)

test_mse = np.mean(
    (y_test - test_prediction)**2
)

df_eff = 1 + np.sum(
    singular_values**2 /
    (singular_values**2 + n * lam)
)

final_results.append([
    name,
    lam,
    train_mse,
    test_mse,
    df_eff
])

final_results = pd.DataFrame(
    final_results,
    columns=[
        "model",
        "lambda",
        "train_mse",
        "test_mse",
        "df_eff"
    ]
)

```

In [ ]:

In [ ]:

In [ ]: